Show me python code for this project

Below is a complete, runnable Python project skeleton for **Support Triage Intelligence**. It uses Laya's `Router` to make typed decisions—category, urgency, escalation/churn risk, and routing—then evaluates accuracy, latency, throughput, and automation rate. Laya supports `choice`, `score`, and `noul` decisions in a single forward pass and provides `predict_batch` for high-throughput processing.[1]

1. Install dependencies

```
python -m pip install laya pandas scikit-learn matplotlib
```

Laya requires Python 3.10 or newer; its `Router` automatically selects an available CUDA, MPS, XPU, or CPU device.[1]

2. Project structure

```
laya-support-triage/
├── data/
│   └── tickets.csv
├── outputs/
├── triage.py
└── evaluate.py
```

Your `data/tickets.csv` should contain at least:

```
ticket_id,text,true_category,true_priority,true_escalation,true_route
1,"I was charged twice and cannot access my account.",billing,high,yes,billing
2,"The app crashes when I open settings.",technical,medium,no,technical
```

3. Define the Laya decision schema

Save this as `triage.py`:

```
import time
import json
from pathlib import Path

import pandas as pd
from laya import Router
```

```
DATA_PATH = Path("data/tickets.csv")
OUTPUT_PATH = Path("outputs/laya_triage_results.csv")
METRICS_PATH = Path("outputs/laya_triage_metrics.json")

# Laya questions: choice = one label, score = ordinal level, noul = P(true)
TRIAGE_QUESTIONS = {
    "category": {
        "type": "choice",
        "instructions": "What is the primary issue in this support ticket?",
        "criteria": {
            "billing": "Payments, invoices, refunds, charges, subscriptions.",
            "technical": "Bugs, errors, outages, login or product failures.",
            "account": "Account access, password, profile, permissions.",
            "product_request": "Feature requests, pricing questions, upgrades.",
            "cancellation": "Cancellation, downgrade, contract termination."
        }
    },
    "urgency": {
        "type": "score",
        "instructions": "How urgent is this ticket?",
        "criteria": [
            "not urgent",
            "soon",
            "important",
            "high business impact",
            "critical or blocking"
        ]
    },
    "escalation": {
        "type": "noul",
        "instructions": "Does this ticket require escalation to a senior human agent
    },
    "churn_risk": {
        "type": "noul",
        "instructions": "Does the customer threaten to cancel, leave, or churn?"
    },
    "security_risk": {
        "type": "noul",
        "instructions": "Does this ticket indicate account compromise, phishing, or u
    },
    "route": {
        "type": "choice",
        "instructions": "Which team should handle this ticket?",
        "criteria": {
            "billing": "Billing, payments, invoices, refunds.",
            "technical": "Technical support and engineering escalations.",
            "account_management": "Account access and profile issues.",
            "security": "Fraud, phishing, compromise, unauthorized access.",
            "retention": "Cancellation, churn risk, retention offers."
        }
    }
}


def extract_answer(result: dict, question_id: str) -> dict:
    """Normalize a Laya answer into a simple dictionary."""
```

```
    answer = result["answers"][question_id]

    if answer["type"] == "choice":
        return {
            "value": answer["choice"],
            "confidence": answer.get("confidence"),
            "probabilities": answer.get("probabilities", {})
        }

    if answer["type"] == "score":
        return {
            "value": answer["score"],
            "confidence": answer.get("confidence"),
            "probabilities": answer.get("probabilities", {})
        }

    # noul returns probability that the answer is true
    return {
        "value": bool(answer["noul"] >= 0.5),
        "probability": answer["noul"],
        "confidence": answer.get("confidence")
    }


def analyze_ticket(router: Router, text: str) -> dict:
    start = time.perf_counter()
    result = router.predict({"body": text}, TRIAGE_QUESTIONS)
    elapsed_ms = (time.perf_counter() - start) * 1000

    row = {"latency_ms": elapsed_ms}

    for question_id in TRIAGE_QUESTIONS:
        extracted = extract_answer(result, question_id)
        row[f"{question_id}_prediction"] = extracted["value"]

        if question_id in {"escalation", "churn_risk", "security_risk"}:
            row[f"{question_id}_probability"] = extracted["probability"]
        else:
            row[f"{question_id}_confidence"] = extracted["confidence"]

    row["routed_model"] = result["routing"]["model"]
    return row


def run_triage():
    df = pd.read_csv(DATA_PATH)

    # Use Router for mixed-language traffic; preload for stable benchmarking.
    router = Router(preload=True)

    records = []
    for _, ticket in df.iterrows():
        analysis = analyze_ticket(router, str(ticket["text"]))
        analysis["ticket_id"] = ticket["ticket_id"]
        records.append(analysis)
```

```
    results = pd.concat([df, pd.DataFrame(records)], axis=1)
    OUTPUT_PATH.parent.mkdir(exist_ok=True)
    results.to_csv(OUTPUT_PATH, index=False)

    print(f"Processed {len(results)} tickets.")
    print(f"Saved predictions to {OUTPUT_PATH}")
    return results


if __name__ == "__main__":
    run_triage()
```

The question schema mirrors Laya's documented API: a `choice` selects one label, a `score` returns an expected ordinal level, and `noul` returns the probability that a yes/no statement is true.[1]

4. Evaluate performance

Save this as `evaluate.py`:

```
import json
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    mean_absolute_error,
    confusion_matrix,
    roc_auc_score
)

RESULTS_PATH = Path("outputs/laya_triage_results.csv")
METRICS_PATH = Path("outputs/laya_triage_metrics.json")
CHART_PATH = Path("outputs/latency_distribution.png")


def classification_metrics(df, truth_col, prediction_col, labels=None):
    y_true = df[truth_col].astype(str)
    y_pred = df[prediction_col].astype(str)

    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "macro_f1": f1_score(y_true, y_pred, average="macro", labels=labels),
        "confusion_matrix": confusion_matrix(y_true, y_pred, labels=labels).tolist()
    }


def binary_metrics(df, truth_col, probability_col, threshold=0.5):
    y_true = df[truth_col].astype(str).str.lower().eq("yes").astype(int)
    y_prob = df[probability_col].astype(float)
    y_pred = (y_prob >= threshold).astype(int)
```

```
    metrics = {
        "accuracy": accuracy_score(y_true, y_pred),
        "f1": f1_score(y_true, y_pred)
    }

    # ROC-AUC is only meaningful when both classes occur.
    if y_true.nunique() == 2:
        metrics["roc_auc"] = roc_auc_score(y_true, y_prob)

    return metrics


def evaluate():
    df = pd.read_csv(RESULTS_PATH)

    category_labels = [
        "billing", "technical", "account",
        "product_request", "cancellation"
    ]

    route_labels = [
        "billing", "technical", "account_management",
        "security", "retention"
    ]

    metrics = {
        "category": classification_metrics(
            df, "true_category", "category_prediction", category_labels
        ),
        "route": classification_metrics(
            df, "true_route", "route_prediction", route_labels
        ),
        "urgency_mae": mean_absolute_error(
            df["true_priority"].astype(float),
            df["urgency_prediction"].astype(float)
        ),
        "escalation": binary_metrics(
            df, "true_escalation", "escalation_probability"
        ),
        "churn_risk": binary_metrics(
            df, "true_escalation", "churn_risk_probability"
        ),
        "latency_ms": {
            "mean": df["latency_ms"].mean(),
            "median": df["latency_ms"].median(),
            "p95": df["latency_ms"].quantile(0.95)
        },
        "throughput_tickets_per_second": 1000 / df["latency_ms"].mean(),
        "automation_rate": (
            (df["escalation_probability"] < 0.5)
            & (df["churn_risk_probability"] < 0.5)
            & (df["security_risk_probability"] < 0.5)
        ).mean()
    }

    METRICS_PATH.write_text(json.dumps(metrics, indent=2))
```

```
    print(json.dumps(metrics, indent=2))

    plt.figure(figsize=(8, 5))
    plt.hist(df["latency_ms"], bins=30)
    plt.xlabel("Latency per ticket (ms)")
    plt.ylabel("Number of tickets")
    plt.title("Laya support-triage latency distribution")
    plt.tight_layout()
    plt.savefig(CHART_PATH)

    return metrics


if __name__ == "__main__":
    evaluate()
```

5. Run the project

```
python triage.py
python evaluate.py
```

The first script writes `outputs/laya_triage_results.csv`; the second writes metrics and a latency chart.

6. Optional: batch mode

For larger datasets, replace the per-ticket loop with Laya's batch API. Laya's documentation notes that compatible requests can share forward passes, improving throughput.[1]

```
def run_triage_batch(df: pd.DataFrame, router: Router, batch_size: int = 8):
    requests = [
        {"state": {"body": str(text)}, "questions": TRIAGE_QUESTIONS}
        for text in df["text"]
    ]

    start = time.perf_counter()
    results = router.predict_batch(
        requests,
        batch_size=batch_size,
        sort_by_length=True
    )
    total_seconds = time.perf_counter() - start

    records = []
    for result in results:
        row = {}
        for question_id in TRIAGE_QUESTIONS:
            extracted = extract_answer(result, question_id)
            row[f"{question_id}_prediction"] = extracted["value"]

            if question_id in {"escalation", "churn_risk", "security_risk"}:
                row[f"{question_id}_probability"] = extracted["probability"]
            else:
```

```
                row[f"{question_id}_confidence"] = extracted["confidence"]

        records.append(row)

    batch_df = pd.DataFrame(records)
    batch_df["batch_total_seconds"] = total_seconds
    batch_df["tickets_per_second"] = len(df) / total_seconds
    return batch_df
```

7. Make the report stronger

Add these comparisons to demonstrate Laya's value rather than only reporting its raw scores:

- **Keyword baseline:** flag tickets containing words such as "refund," "urgent," "cancel," or "fraud."
- **Classical ML baseline:** TF-IDF + logistic regression for category classification.
- **Confidence threshold:** route only tickets above a chosen confidence threshold to automation; send the rest to humans.
- **Cost analysis:** compare local Laya inference cost with an external generative-LLM API.
- **Ablation:** run Laya's base checkpoint versus its `typed-decisions` checkpoint, since Laya reports substantially higher benchmark accuracy for the fine-tuned typed-decisions checkpoint. [1]

For a polished dashboard, load `outputs/laya_triage_results.csv` into Streamlit or Power BI and display ticket volume, category mix, urgency distribution, escalation queue, churn-risk cases, latency, and percentage of tickets eligible for no-touch automation.

⁂

1. https://github.com/NandhaKishorM/laya/blob/main/README.md